On-Prem to Azure Ingestion with Data Factory and SHIR

Nov 1, 2026

What you're building

A Data Factory pipeline picks up every `*.csv` file in `C:\outbox` on your own Windows machine, uploads it to the `landing` container in the data lake, and then moves the original into `C:\outbox\archive`. The machine is reached through a **self-hosted integration runtime (SHIR)**: a Windows service that only makes outbound HTTPS connections to Azure, so no inbound ports are opened.

```
Your laptop (Windows)                        Azure
+---------------------------+               +--------------------------------------+
| C:\outbox\*.csv           |               | Data Factory (managed identity)      |
| C:\outbox\archive\        |<--- SHIR ----->|  +- ls_outbox   (FileServer via SHIR)|
| SHIR (Integration Runtime)|  outbound 443  |  +- ls_datalake (ADLS, managed ID)   |
+---------------------------+               |  +- ls_keyvault (password for outbox)|
                                            |  +- pipeline: list > copy > archive  |
                                            | Storage: landing/   Key Vault        |
                                            +--------------------------------------+
```

Data Factory sends the SHIR instructions ("list `C:\outbox`", "read this file"); the SHIR runs them locally and streams the data up.

The pipeline: `pl_outbox_to_landing`

```
ListOutbox (Get Metadata: childItems of C:\outbox)
   |
   v
OnlyCsv (Filter: type = File AND name ends with .csv)
   |
   v
EachFile (ForEach, up to 4 files in parallel)
   |
   +--> CopyToLanding : C:\outbox\<file>  -->  landing/<file>
            |
            | only on Succeeded
            v
        ArchiveSource : C:\outbox\<file>  -->  C:\outbox\archive\<file>
                        then delete C:\outbox\<file>   (= a move)
```

Why the design looks like this

- **List the files, then handle each one.** A single wildcard copy followed by a wildcard delete could delete a file that arrived between the two steps without ever uploading it. This way, only files that were actually copied get archived.
- **The archive step only runs after a successful upload.** If the upload fails, the file stays in `C:\outbox` and is retried on the next run.
- **No secrets in Terraform.** The password lives only in Key Vault. ADF fetches it at run time using its managed identity.

Where it lives in the repo

The assignment asks for Bicep; this project uses Terraform, so the factory is a Terraform module. The `landing` and `archive` containers already exist in `modules/storage`.

File	What it holds
`infra/modules/datafactory/main.tf`	Factory (system-assigned identity), 2 role assignments, SHIR, 3 linked services, 4 datasets, the pipeline
`infra/modules/datafactory/variables.tf`	Inputs, incl. `outbox_path`, `outbox_user`, `outbox_password_secret_name`
`infra/modules/datafactory/outputs.tf`	`name`, `id`, `principal_id`, `shir_name`, `shir_auth_key` (sensitive)
`infra/modules/storage/outputs.tf`	Adds `dfs_endpoint` for the lake linked service
`infra/main.tf`	`module "adf"` wiring
`infra/variables.tf`	`outbox_path` (default `C:\outbox`), `outbox_user`
`infra/env/dev.tfvars`	`outbox_user = "VILSHUB-NET\\adfreader"`

What the module creates:

Resource	Name	Purpose
Data factory	`adf-ingest-<env>-<suffix>`	Runs the pipeline; has a system-assigned managed identity
Role assignment	Storage Blob Data Contributor on the storage account	Factory can write to `landing`
Role assignment	Key Vault Secrets User on the vault	Factory can read the outbox password
Integration runtime	`shir-onprem`	The self-hosted runtime your laptop registers to
Linked service	`ls_keyvault`	Points at the Key Vault
Linked service	`ls_datalake`	ADLS Gen2, signs in with the managed identity
Linked service	`ls_outbox`	FileServer at `C:\outbox`, connects via `shir-onprem`, password from Key Vault
Datasets	`ds_outbox_folder`, `ds_outbox_file`, `ds_outbox_archive`, `ds_landing_file`	Binary datasets (files copied byte for byte); the file ones take a `fileName` parameter
Pipeline	`pl_outbox_to_landing`	The flow above

Step 0: Prerequisites (once)

1. Register the Data Factory provider. Same reason as Key Vault in the OIDC guide: the pipeline identity cannot register providers.

- Portal: Subscriptions, your subscription, Settings, Resource providers, filter `Microsoft.DataFactory`, select it, Register.
- CLI:

```
az provider register -n Microsoft.DataFactory --wait
```

2. Let the pipeline identity create role assignments. The module grants the factory access to storage and Key Vault. The pipeline identity is only Contributor on `rg-ingest-dev`, and Contributor cannot create role assignments, so the apply fails with `AuthorizationFailed ... roleAssignments/write`.

- Portal: `rg-ingest-dev`, Access control (IAM), Add role assignment, **Privileged administrator roles** tab, Role Based Access Control Administrator, Members: select the app registration, Review + assign.
- CLI:

```
az role assignment create --assignee 4b0e464d-9fa9-4540-9095-853f79bdf0b7 \
  --role "Role Based Access Control Administrator" \
  --scope $(az group show -n rg-ingest-dev --query id -o tsv)
```

3. Create a dedicated Windows account and the folders. Do not use your personal login password. PowerShell as Administrator:

```
net user adfreader "<StrongPassword>" /add
mkdir C:\outbox\archive
icacls C:\outbox /grant "adfreader:(OI)(CI)M"
```

Step 1: Deploy the Terraform

Push to the `dev` branch. The `infra` workflow runs `terraform apply` and creates everything in the table above. Check in the portal that `adf-ingest-dev-...` exists in `rg-ingest-dev`.

Step 2: Put the password in Key Vault (manual, once)

Terraform never holds the password (it would end up in the state file). The vault uses RBAC, so first give yourself **Key Vault Secrets Officer** on it (vault, Access control (IAM), Add role assignment).

- Portal: the vault, **Objects**, **Secrets**, **+ Generate/Import**, Name `outbox-password`, Secret value = the `adfreader` password, **Create**.
- CLI:

```
az keyvault secret set --vault-name <kv-ingest-dev-...> -n outbox-password --value '<adfreade
r password>'
```

Step 3: Install and register the SHIR

1. Get the authentication key, either way:
   - Portal: Data Factory, **Launch Studio**, **Manage** (toolbox icon), **Integration runtimes**, `shir-onpre m`, copy **Key1**.
   - Terraform: `terraform output -raw shir_auth_key`.
2. Download the installer: search "Microsoft Integration Runtime download" or open `https://www.micro soft.com/download/details.aspx?id=39717`, pick the latest `IntegrationRuntime_5.x.msi`.
3. Run it on **Windows** (not inside WSL), keep the defaults, **Finish**.
4. Configuration Manager opens: paste the key, **Register**, **Finish**.
5. In ADF Studio, Manage, Integration runtimes: `shir-onprem` shows **Running**.
6. Newer SHIR versions block local paths like `C:\outbox` by default. If a run fails with a "local folder path" / "not allowed" error, PowerShell as Administrator:

```
cd "C:\Program Files\Microsoft Integration Runtime\5.0\Shared"
.\dmgcmd.exe -DisableLocalFolderPathValidation
```

Step 4: Test it

1. Copy `samples/orders.csv` from the repo into `C:\outbox\`.

2. ADF Studio, **Author**, Pipelines, `pl_outbox_to_landing`, **Debug** (or **Add trigger**, **Trigger now**).
3. Check:
   - Storage account, `landing` container: `orders.csv` is there.
   - `C:\outbox\orders.csv` is gone; `C:\outbox\archive\orders.csv` exists.
   - **Monitor** tab: the run is green and every activity succeeded.

Troubleshooting

Error	Fix
`MissingSubscriptionRegistration ... Microsoft.DataFactory`	Step 0.1
`AuthorizationFailed ... roleAssignments/write` during apply	Step 0.2
Linked service test: `Access denied` / `Logon failure`	`outbox_user` must be `HOSTNAME\adfreader`; the Key Vault secret must match the password; the account needs rights on `C:\outbox`
`Forbidden` reading the secret	Factory identity lacks Key Vault Secrets User (Terraform adds it; allow a few minutes to propagate)
`403` writing to `landing`	Storage Blob Data Contributor for the factory has not propagated yet; wait and re-run
Local folder path not allowed	Step 3.6
SHIR shows **Unavailable**	Laptop off or asleep, or the service stopped; open Configuration Manager

Things to know

- **Same file name overwrites.** A second `orders.csv` replaces the first one in `landing`. To keep both, give the landing dataset a date folder, e.g. `folderPath = "@formatDateTime(utcNow(),'yyyy/MM/dd')"`.
- **No schedule yet.** The pipeline runs only when triggered. Add `azurerm_data_factory_trigger_schedule` to run it every N minutes.
- **One SHIR, one factory.** A SHIR install registers to a single data factory. The laptop serves `dev`; prod needs its own SHIR machine.